

Smart Asset Management and Resource Allocation Platform

Design Document

Cultural Council, IIT Roorkee — Cult Open Projects 2026

1. Problem Understanding

Organizations and institutions often manage a large number of shared resources such as cameras, audio systems, recording equipment, costumes, stage props, and event infrastructure. Traditional inventory management methods based on spreadsheets and manual coordination lead to inefficiencies, scheduling conflicts, and lack of visibility.

The Cultural Council of IIT Roorkee faces this challenge while managing a large pool of shared resources including DSLR cameras, studio lighting equipment, audio systems, costumes, stage props, recording equipment, and event infrastructure. These assets are frequently shared across multiple sections and events, making efficient resource allocation critical.

The objective of this project is to build a centralized full-stack platform that enables efficient inventory tracking, resource allocation, booking workflows, approval mechanisms, and operational analytics. The platform provides role-based access for administrators and resource consumers while maintaining data consistency, simplicity, usability, and scalability.

2. System Architecture

The system follows a client-server architecture with a clear separation between frontend, backend, and database layers.

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- React Query
- Axios

Backend

- Node.js
- Express.js
- TypeScript

Database

- PostgreSQL (Neon)
- Prisma ORM

Deployment

- Frontend: Vercel
- Backend: Render
- Database: Neon PostgreSQL

Request Workflow

User -> Frontend (React) -> REST API (Express) -> Prisma ORM -> PostgreSQL Database

The frontend communicates with the backend exclusively through a versioned REST API. The backend handles authentication, authorization, business logic, and validation, while Prisma provides a type-safe data access layer to the PostgreSQL database. This modular separation ensures maintainability and allows each layer to scale independently.

3. Database Schema

User — Stores user information and authentication details.

- id (PK)
- name
- email
- passwordHash
- role
- createdAt

Category — Stores asset categories.

- id (PK)
- name
- description

Asset — Stores inventory information.

- id (PK)
- name
- description
- categoryId (FK -> Category)
- totalQuantity
- availableQuantity
- status
- condition

Booking — Stores booking requests and approval information.

- id (PK)
- userId (FK -> User)
- assetId (FK -> Asset)
- quantity
- startDate
- endDate
- status

Notification — Stores notifications sent to users.

- id (PK)
- userId (FK -> User)
- message
- isRead
- createdAt

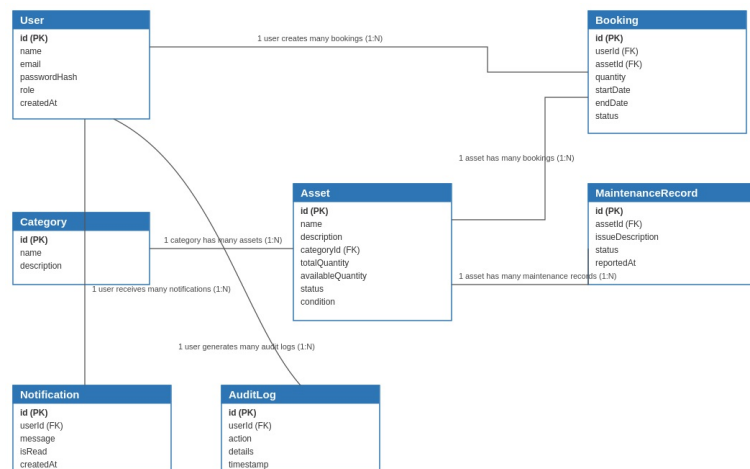
AuditLog — Tracks system activities.

- id (PK)
- userId (FK -> User)
- action
- details
- timestamp

MaintenanceRecord — Stores maintenance and damage information.

- id (PK)
- assetId (FK -> Asset)
- issueDescription
- status
- reportedAt

4. Entity Relationship Diagram (ERD)



Entity Relationship Diagram

Relationships

- One User can create multiple Bookings.
- One Asset belongs to one Category.
- One Asset can have multiple Bookings.
- One Asset can have multiple Maintenance Records.
- One User can receive multiple Notifications.
- One User can generate multiple Audit Logs.

5. API Overview

Authentication APIs

- POST /api/auth/register
- POST /api/auth/login

Asset APIs

- GET /api/assets
- POST /api/assets
- PUT /api/assets/:id
- DELETE /api/assets/:id

Booking APIs

- GET /api/bookings
- POST /api/bookings
- PUT /api/bookings/:id

Category APIs

- GET /api/categories
- POST /api/categories

Analytics APIs

- GET /api/analytics

Notification APIs

- GET /api/notifications

Audit APIs

- GET /api/audit

Maintenance APIs

- GET /api/maintenance
- POST /api/maintenance

All endpoints (except authentication routes) are protected via JWT-based authentication middleware, with role-based access control restricting administrative operations such as asset creation, deletion, and booking approval to admin users.

6. Design Decisions

- **Prisma ORM** was chosen for type-safe database interaction, reducing runtime errors and improving developer productivity through auto-generated types and migrations.
- **PostgreSQL** was selected for its relational integrity, scalability, and reliability in handling structured inventory and booking data with strong consistency guarantees.
- **JWT-based authentication** ensures secure, stateless session management and simplifies horizontal scaling of the backend.
- **React Query** improves API state management, caching, and synchronization between the frontend and backend, reducing unnecessary network calls.
- **Modular backend architecture** (controllers, services, routes, middleware) improves maintainability and allows independent development and testing of features.
- **Role-based access control** cleanly separates administrator

capabilities (inventory management, approvals, analytics) from regular user capabilities (browsing, booking, tracking history).

- **Booking validation logic** prevents requests that exceed available inventory, ensuring data integrity is maintained throughout the asset lifecycle.
- **Vercel + Render + Neon** deployment stack was chosen for ease of continuous deployment, free-tier availability, and minimal DevOps overhead, while keeping the system production-ready and publicly accessible.

Future Improvements

- Email-based notification service
- QR-code scanning for issue/return operations
- Asset health prediction using maintenance history
- Advanced analytics dashboard (utilization trends, overdue forecasting)
- Multi-organization / multi-tenant support
- Real-time notifications via WebSockets